

Assignment 1

Sree Venkata Anjananand Iragavarapu

a1987826

27 August 2026

1. Implementation

A multilayer perceptron was implemented in NumPy: 4 inputs, 16 hidden units with ReLU activation, and 3 output units with a softmax output layer. The forward propagation, backward propagation, and SGD update step were implemented manually. Scikit-learn was only used to load and preprocess the data.

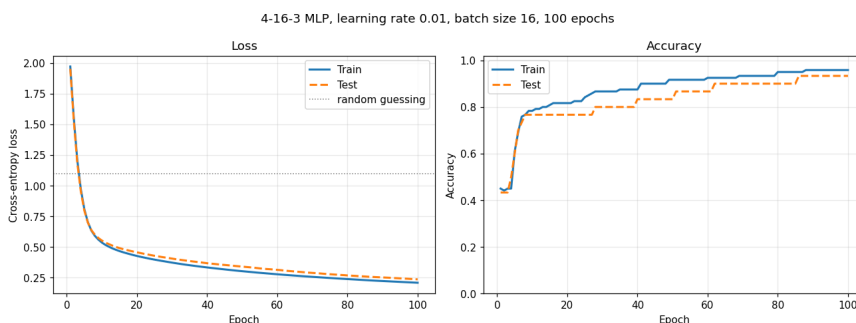
Preprocessing. Instead of normalizing the input features to lie within a particular range, they were standardized, as the range of sepal length varies from 4 to 8 cm and the range for petal width varies from 0.1 to 2.5, and a single learning rate would overstep in one direction and crawl in another direction. The normalizer was fit on the training set only, hence no information from the test set leaks into training. The split was performed in a stratified way to ensure that there are exactly 10 test samples per class, and the labels were one-hot encoded.

Numerical stability. The softmax function uses subtraction of the row maximum prior to exponentiation, which does not affect the final result but prevents overflow to infinity. The loss clips the predicted probabilities to prevent $\log(0)$.

Backward pass. The combined expression for the softmax function and the cross-entropy loss simplifies to the difference between the predicted probabilities and the true label, divided by the batch size. The gradient for weights is the result of matrix multiplication of the transpose of the activation matrix by the gradient matrix. The hidden units receive their gradient scaled by the ReLU derivative.

Training. The weights had He initialization while the biases started from 0. The network was trained using the mini-batch SGD algorithm with a batch size of 16, learning rate of 0.01, and 100 epochs with reshuffling. In total, there are 800 iterations.

2. Results and Analysis



Epoch	Loss	Train acc	Actual	Set.	Vers.	Virg.	P	R	F1
1	1.9708	0.4500	Setosa	10	0	0	1.000	1.000	1.000
50	0.3023	0.9167	Versicolor	0	8	2	1.000	0.800	0.889
100	0.2083	0.9583	Virginica	0	0	10	0.833	1.000	0.909

The test accuracy was 93.33 per cent, that is 28 out of 30, surpassing the 90 per cent target. The backpropagation was compared with the central-difference numerical gradients before training, providing the worst relative error of 3.57×10^{-8} .

Error analysis. The Setosa was classified perfectly due to its separability by the petal measurements only. Both errors are that Versicolor was predicted as Virginica, never vice versa, showing up as perfect precision but 0.800 recall for Versicolor versus perfect recall but 0.833 precision for Virginica. The network was only 0.613 and 0.642 confident on those two predictions, so the model is uncertain precisely where the two classes genuinely overlap, rather than being confidently wrong.

Convergence. The test loss follows the training loss with no divergence, and therefore there is no overfitting. Although the loss is still decreasing at epoch 100 and the training accuracy stops at 0.9583, the model has not fitted even the training data. This is called underfitting due to the update budget: 800 iterations of normal SGD with a learning rate of 0.01 is not sufficient for convergence. Extended training, increasing the learning rate, or employing an adaptive algorithm such as Adam will solve the problem, but not a bigger model.

3. Problems Encountered and Resolutions Adopted

The problem of backpropagation failures is resolved by dividing by batch size and still computing a descending loss curve, although not the right one. To guard against this, gradients were verified numerically before any training by perturbing each parameter up and down and comparing the computed slope with the analytical one. The largest relative error found was 3.57×10^{-8} .

A false alarm in the above verification. The first implementation failed at 2×10^{-5} , implying a broken backward pass. The failing case turned out to have gradient 4×10^{-6} , matching the reported value to ten significant figures. As the gradient gets small, the loss values used in the subtraction get similar enough that the floating point cancellation makes the difference meaningless, hence the reported large relative error. Entries like these are now evaluated for their absolute error.

Softmax overflow; For inputs larger than approximately 709, the exponential function returns infinity, and the division becomes meaningless, propagating silently into the remaining computation. The problem is solved by subtracting the maximum row, as shown in the notebook with the input [1000, 1, 1].

Proposed improvements. Because the problem involves underfitting rather than overfitting, the best option will be to substitute stochastic gradient descent with Adam optimizer and momentum as both will have a much greater coverage in the same number of 800 steps. It may also be useful to try L_2 weight decay, but since there is no overfitting in the current graph, it will mostly help to keep the decision boundary from becoming overly sharp. Grid search on hidden layer sizes, learning rates, and number of epochs should determine whether the problem lies in capacity or in optimization. Finally, it is better to rerun the experiment several times using different random seeds and report results as mean and its standard deviation, because each test sample is worth 3.33 percentage points.

4. Answer to the Questions

Q1. What is the reason for choosing ReLU activation function for the hidden layer instead of sigmoid or linear?

If I use a linear activation function in the hidden layer, then the hidden layer loses its meaning completely. Stacking two linear functions mathematically is equal to having just one layer, since I can multiply the two weight matrices to have a combined matrix. The whole point of the extra layer is the nonlinearity.

The gradient of the sigmoid function reaches a maximum of 0.25 around the middle of its range, and as the units approach their extremes, the function goes to nothing. In contrast to sigmoid, with ReLU, the earlier layers learn from their mistakes. It is also more efficient.

In case of using just one hidden layer, the shrinking takes place just once and not repeatedly. Therefore, instead of being broken, sigmoid is likely to be slow here. Any unit that gets negative values for all possible inputs has a gradient of zero and will never recover.

Q2. In case a model is not performing well in test data, what are the two reasons and what would be solutions for each of them?

The first reason could be overfitting. This means that the model has learned not only the pattern but also the noise from the training data. The solutions are adding weight decay regularization to penalize larger weights, stopping training at the peak of the test performance curve, reducing the number of hidden units, using dropout or collecting more data.

The second reason could be underfitting, meaning that the model has not yet learned enough from the training data, being too simple for the task or receiving insufficient iterations from the optimizer. In this case, the solution would be to train longer, increase the learning rate, increase the capacity or use adaptive optimizers such as Adam.

Statement on the Use of Generative AI

I used both Gemini and Claude to conduct research and develop the outline of this project. Furthermore, I used these two AI tools to help me understand the math behind backpropagation and numerical gradient checking. They helped me in polishing the code and providing language editing, feedback according to the rubric of evaluation and also in latex formatting and typesetting of the report. All final content, calculations are my own.